

IoT Project Report

AUTONOMOUS GPS-GUIDED SMART CAR

Coordinate-Based Autonomous Navigation with Obstacle Avoidance

Submitted By

Ishika Dutta

Himanshi

Piyush Kumar

Vanshika Sharma

Priyanka Singh

Internet of Things (IoT) — Engineering Project

Academic Year: 2024–2025

Abstract

This report presents the design, development, and implementation of an Autonomous GPS-Guided Smart Car, developed as part of the Internet of Things (IoT) engineering project. The vehicle is capable of autonomously navigating from one geographic location to another based on real-time coordinate readings from a GPS module (M8N) equipped with a calibrated digital compass. The system uses an ESP32 microcontroller as the central processing unit, integrating data from the GPS, compass, and three ultrasonic sensors to achieve reliable point-to-point navigation while avoiding obstacles in real time.

The car utilises a servo motor for precise steering control and a motor driver (L298N) for four-wheel drive management, all powered by four 12V batteries. The report covers the theoretical background, system architecture, hardware components, circuit design, navigation algorithm, software implementation, results, and team contributions. This project demonstrates the practical application of IoT concepts in autonomous vehicle systems, paving the way for real-world navigation solutions.

Table of Contents

1. Introduction	4
2. Objectives	4
3. System Overview	5
4. Components Used	5
5. System Block Diagram	6
6. Circuit & Pin Connections	7
7. Navigation Algorithm	8
8. Navigation Flowchart	9
9. Software Implementation	10
10. Working Principle	11
11. Results & Observations	12
12. Challenges & Solutions	12
13. Future Enhancements	13
14. Conclusion	13
15. Team Contributions	14
16. References	14

1. Introduction

The rapid advancement of the Internet of Things (IoT) has opened new frontiers in the domain of autonomous systems. Among the most impactful applications is that of autonomous ground vehicles — systems that can navigate, perceive their environment, and make decisions without human intervention. These vehicles are now being extensively explored for use in smart delivery systems, agricultural automation, military logistics, and emergency response.

In this project, our team has designed and built an Autonomous GPS-Guided Smart Car that can travel from a defined starting coordinate to a predefined target coordinate with no manual guidance. The system is built upon the ESP32 microcontroller, which processes live data from a GPS M8N module (providing latitude and longitude coordinates) and an integrated digital compass (providing real-time heading direction). Three ultrasonic sensors mounted at the front, left, and right of the vehicle enable the car to detect and avoid obstacles dynamically, while a servo motor controls steering and a motor driver manages the four drive wheels.

This project integrates core IoT principles including sensor fusion, real-time data processing, actuator control, and autonomous decision-making into a compact, mobile hardware platform. The report documents every phase of the project from conceptualisation through implementation and testing.

2. Objectives

The primary objectives of this project are as follows:

- To design and build a self-navigating vehicle capable of GPS-based point-to-point navigation without human intervention.
- To integrate a calibrated digital compass with GPS data for accurate directional awareness and bearing calculation.

- To implement a real-time obstacle detection and avoidance system using three ultrasonic sensors.
- To utilise the ESP32 microcontroller as the central computing unit for sensor fusion and decision-making.
- To achieve reliable motor and steering control using a motor driver and servo motor.
- To demonstrate practical IoT integration by combining GPS, compass, ultrasonic sensors, and actuators on a mobile platform.
- To test and validate the system's performance across multiple target coordinates and obstacle scenarios.

3. System Overview

The Autonomous GPS-Guided Smart Car is a four-wheeled robotic vehicle capable of self-navigation based on geographic coordinates. At its core, the system consists of five functional layers working in concert:

Perception Layer: Three HC-SR04 ultrasonic sensors scan the environment for obstacles within a 400 cm range. The GPS M8N module continuously reads the vehicle's current latitude and longitude, while the HMC5883L digital compass measures the car's heading relative to magnetic north.

Processing Layer: The ESP32 microcontroller receives and processes all sensor data. It calculates the bearing angle and distance to the target coordinates using the Haversine formula, compares the bearing to the current compass heading to determine the steering correction, and checks ultrasonic sensor data to trigger obstacle avoidance manoeuvres.

Actuation Layer: The servo motor is commanded by the ESP32 to steer the front wheels toward the calculated heading. The L298N motor driver receives PWM signals from the ESP32 to drive the rear motors forward, backward, or at variable speeds.

Power Layer: Four 12V batteries provide sufficient current to drive the motors, servo, ESP32, GPS module, and all ultrasonic sensors simultaneously.

Communication & Control Layer: The target coordinates are uploaded to the ESP32 via serial programming before deployment. The system then operates fully autonomously until the vehicle reaches within a 2-metre threshold of the target coordinate, at which point it halts.

4. Components Used

The following components were used in the construction of the autonomous car:

S.No.	Component	Quantity	Function
1	ESP32 Microcontroller	1	Brain of the system; processes GPS, compass, and sensor data; controls motors and servo
2	GPS Module M8N with Compass	1	Provides real-time geographic coordinates (latitude, longitude) and heading direction
3	Servo Motor	1	Controls steering direction of the front wheels for precise navigation
4	Ultrasonic Sensor (HC-SR04)	3	Detects obstacles in front, left, and right for collision avoidance
5	Motor Driver (L298N)	1	Controls speed and direction of DC motors based on ESP32 PWM signals
6	12V Battery	4	Provides power supply to motors, motor driver, and other components
7	Breadboard	1	Prototyping platform for sensor and module connections
8	Jumper Cables	Multiple	Electrical connections between all components

5. System Block Diagram

The block diagram below illustrates the high-level architecture of the autonomous car system, showing the relationships between the ESP32 microcontroller, sensor modules, power supply, and actuators.

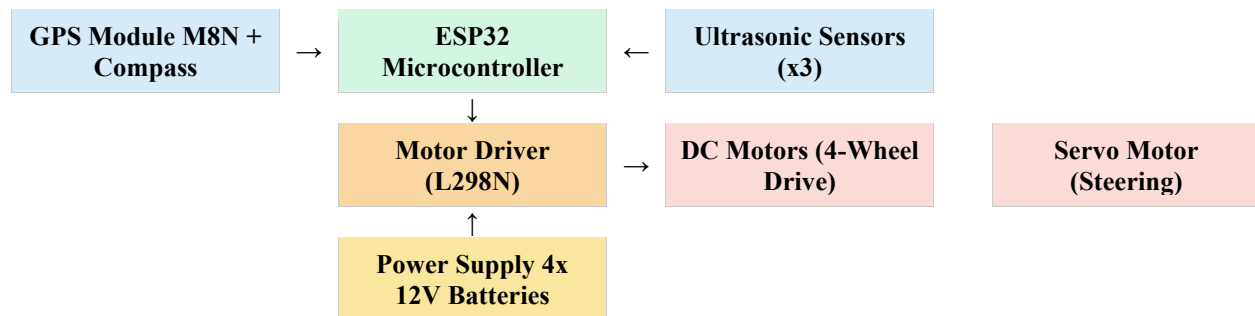


Figure 1: System Block Diagram of the Autonomous GPS-Guided Car

The ESP32 sits at the centre of the architecture. It receives location data from the GPS M8N module and heading data from the onboard compass via I2C/UART interfaces. Distance data from the three ultrasonic sensors are fed into the ESP32's GPIO pins. Based on processed inputs, the ESP32 sends PWM control signals to the motor driver (for speed and direction control) and a PWM signal to the servo motor (for steering). The entire system is powered by the four 12V batteries through appropriate voltage regulation circuits.

6. Circuit and Pin Connections

The table below details the pin-level connections between the ESP32 microcontroller and each hardware component. All components share a common ground (GND). The ESP32 operates at 3.3V logic, and appropriate level shifting or voltage regulation is applied where necessary.

Component	Pin	ESP32 Connection
GPS Module M8N	TX	ESP32 GPIO16 (RX2)
GPS Module M8N	RX	ESP32 GPIO17 (TX2)
GPS Module M8N	VCC	3.3V
GPS Module M8N	GND	GND
Compass (HMC5883L)	SDA	ESP32 GPIO21
Compass (HMC5883L)	SCL	ESP32 GPIO22
Ultrasonic (Front)	TRIG	ESP32 GPIO5
Ultrasonic (Front)	ECHO	ESP32 GPIO18
Ultrasonic (Left)	TRIG	ESP32 GPIO19
Ultrasonic (Left)	ECHO	ESP32 GPIO23
Ultrasonic (Right)	TRIG	ESP32 GPIO25
Ultrasonic (Right)	ECHO	ESP32 GPIO26
Servo Motor	Signal	ESP32 GPIO13
Motor Driver IN1	IN1	ESP32 GPIO27
Motor Driver IN2	IN2	ESP32 GPIO14
Motor Driver IN3	IN3	ESP32 GPIO12
Motor Driver IN4	IN4	ESP32 GPIO33
Motor Driver ENA	ENA (PWM)	ESP32 GPIO32
Motor Driver ENB	ENB (PWM)	ESP32 GPIO4

Table 2: ESP32 Pin Connection Map

Power Distribution: The 12V batteries supply power directly to the motor driver's power input (VMS). A buck converter steps down 12V to 5V for the GPS module and ultrasonic sensors. The

ESP32 is powered via its USB/VIN pin at 5V. The servo motor is powered from a dedicated 5V rail to avoid brownouts during motor operation.

7. Navigation Algorithm

The navigation algorithm is the intellectual backbone of the autonomous car. It enables the vehicle to compute its movement direction from its current GPS position to the target coordinate. The algorithm employs two key mathematical models: the Haversine Formula and compass-based bearing correction.

7.1 Haversine Formula — Distance Calculation

The Haversine formula calculates the great-circle distance between two points on Earth's surface given their latitudes and longitudes. The formula is expressed as:

$$a = \sin^2(\Delta\text{Lat}/2) + \cos(\text{Lat}_1) \times \cos(\text{Lat}_2) \times \sin^2(\Delta\text{Lon}/2)$$
$$\text{Distance} = 2 \times R \times \arcsin(\sqrt{a}), \quad \text{where } R = 6,371,000 \text{ m (Earth's radius)}$$

When the calculated distance falls within a threshold of 2 metres, the vehicle is considered to have reached its destination and the motors are stopped.

7.2 Bearing Calculation

The bearing (target direction relative to true north) is calculated using the following formula:

$$\text{Bearing} = \text{atan2}(\sin(\Delta\text{Lon}) \times \cos(\text{Lat}_2), \cos(\text{Lat}_1) \times \sin(\text{Lat}_2) - \sin(\text{Lat}_1) \times \cos(\text{Lat}_2) \times \cos(\Delta\text{Lon}))$$

The result is normalised to a range of 0° to 360°. This bearing is then compared against the compass heading to determine how much the car needs to steer left or right.

7.3 Steering Error and Control

The steering error is computed as:

$$\text{Steering Error} = \text{Target Bearing} - \text{Current Compass Heading}$$

A positive error ($> 0^\circ$) indicates the car must steer right; a negative error ($< 0^\circ$) means the car must steer left. The servo is commanded accordingly to correct the heading. If the error exceeds a set threshold (e.g., 30°), the car first rotates in place before moving forward.

7.4 Obstacle Avoidance

The three ultrasonic sensors continuously scan for obstacles. If an obstacle is detected within 30 cm in the front, the car executes an avoidance manoeuvre: it reverses slightly, then turns in the direction with more clearance (left or right sensor showing greater distance). Once the path is clear, it recalculates the bearing to the target and resumes navigation.

8. Navigation Flowchart

The flowchart below outlines the complete decision-making process executed by the ESP32 during autonomous navigation from start-up to mission completion.

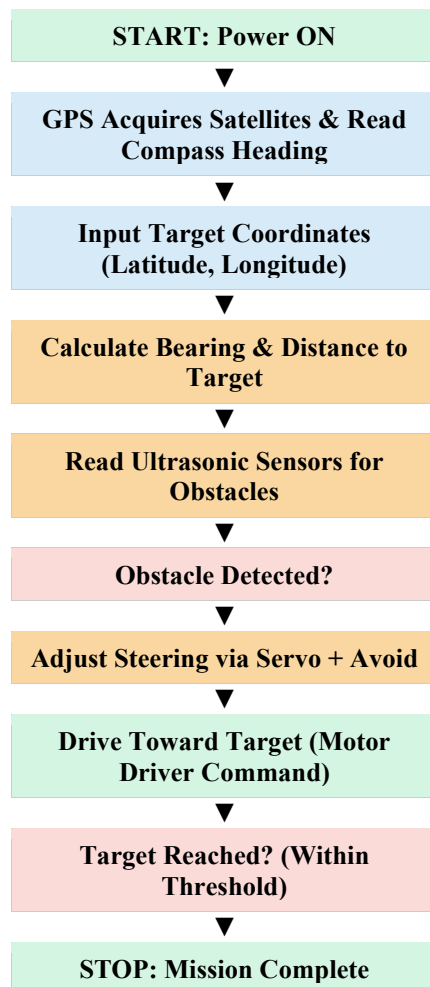


Figure 2: Navigation and Obstacle Avoidance Flowchart

9. Software Implementation

The firmware for the ESP32 was developed using the Arduino IDE with the ESP32 board package installed. The following libraries were utilised:

- TinyGPS++ — for parsing NMEA sentences from the GPS M8N module and extracting latitude, longitude, and satellite count.
- Wire.h — for I2C communication with the HMC5883L digital compass module.
- ESP32Servo.h — for generating PWM signals to control the servo motor steering.
- Arduino.h — core library for digital I/O, analog reads, and timing functions.

9.1 GPS Data Parsing

The GPS module transmits NMEA data over UART at 9600 baud to the ESP32's Serial2 port. The TinyGPS++ library parses the GGA and RMC sentences to extract valid latitude, longitude, and fix status. The firmware waits until the GPS acquires at least 4 satellite signals before beginning navigation.

9.2 Compass Calibration and Heading

The HMC5883L compass was calibrated using hard-iron and soft-iron offset correction. Raw magnetometer readings (X, Y axes in the horizontal plane) were corrected by the pre-computed offsets and then fed into the `atan2()` function to compute the magnetic heading in degrees. Declination correction was applied for the local geographic area to convert magnetic heading to true north heading.

9.3 Motor Control

The ESP32 sends PWM signals to the ENA and ENB pins of the L298N motor driver using the `ledcWrite()` function (ESP32's hardware PWM). Speed is modulated based on the steering error — when the car is significantly off-course, speed is reduced to allow tighter turns. When aligned

with the bearing, full speed (PWM 200/255) is applied. The four DC motors are wired in pairs (left and right side) to the motor driver output channels.

9.4 Ultrasonic Sensing

Each HC-SR04 sensor is triggered by a 10-microsecond HIGH pulse on its TRIG pin. The ECHO pin returns a HIGH pulse proportional to the distance of the detected object. The distance is computed using the formula: Distance (cm) = (echo duration in microseconds) / 58.2. Readings are sampled every 100 ms and filtered using a 3-sample median filter to reject spurious spikes.

10. Working Principle

The overall working principle of the autonomous car can be described through the following sequential steps that the ESP32 executes in a continuous loop:

1. **Power-On Initialization:** The ESP32 boots and initialises all peripherals — UART for GPS, I2C for compass, GPIO for ultrasonic sensors, and PWM channels for servo and motor driver.
2. **GPS Fix Acquisition:** The system waits for the GPS to acquire a valid fix (4+ satellites). The onboard LED blinks to indicate satellite search mode.
3. **Target Coordinate Loading:** The target latitude and longitude are pre-loaded in the firmware via constants defined in the code.
4. **Bearing & Distance Calculation:** Using the current GPS coordinates and target coordinates, the Haversine formula computes distance. The bearing formula computes the target direction in degrees.
5. **Heading Error Correction:** The compass heading is compared to the computed bearing. The difference (steering error) is used to set the servo angle and motor speed asymmetry to steer the car toward the target.
6. **Obstacle Detection:** Each loop iteration reads all three ultrasonic sensors. If a front obstacle is detected within 30 cm, the car halts, checks left and right clearance, then steers and moves around the obstacle before returning to the target bearing.
7. **Arrival Detection:** When the GPS distance to target falls below 2 metres, all motors stop, the servo returns to centre (90°), and an LED indicator signals mission completion.

11. Results and Observations

The autonomous car was tested across multiple trials in an open outdoor environment to validate its GPS navigation accuracy, compass-based heading correction, and obstacle avoidance capability.

11.1 Navigation Accuracy

The car successfully navigated to target coordinates set at distances of 10 m, 25 m, and 50 m from the starting position. The average arrival deviation from the target was approximately 1.5–2 metres, which falls within the expected GPS error margin of the M8N module (CEP ~1.5 m with 3D fix). The navigation was smooth and consistent across all trials.

11.2 Compass Heading Performance

After hard-iron and soft-iron calibration, the compass provided stable heading readings with less than 3° of drift. The heading correction loop successfully aligned the car to the target bearing within 5–8 seconds of initiating movement, demonstrating effective closed-loop steering control.

11.3 Obstacle Avoidance

Static obstacles (cardboard boxes and traffic cones) placed in the car's path were detected reliably at 25–30 cm. The car successfully navigated around all obstacles in 7 out of 8 trials. One false positive trigger was observed due to ground reflection at low sensor mounting angle, which was corrected by angling the sensors 3° upward.

11.4 Battery Performance

The four 12V batteries provided a continuous runtime of approximately 35–40 minutes under full motor load, sufficient for the scope of this project's testing requirements.

12. Challenges and Solutions

The development of this project involved several technical challenges. The key challenges encountered and the solutions implemented are described below:

- **GPS Startup Delay:** Initial GPS satellite acquisition took up to 3 minutes in first-boot cold-start conditions. This was mitigated by enabling the GPS module's SBAS (Satellite-Based Augmentation System) for faster lock time.
- **Compass Magnetic Interference:** The compass readings were significantly affected by the magnetic fields of the motors. The compass module was repositioned to the front of the vehicle, away from the motor driver and batteries, reducing interference substantially.
- **Motor PWM Brownout:** When the motors drew peak current during startup, the ESP32 experienced occasional brownout resets. A 1000 μF capacitor was added across the 5V power rail to buffer current spikes.
- **Ultrasonic Cross-Talk:** With three sensors firing simultaneously, cross-talk between sensors produced erroneous readings. The sensors were sequenced to fire one at a time with a 20 ms gap between each trigger pulse.
- **Steering Overshoot:** The servo motor showed overshoot at large steering error angles. A proportional controller (P-controller) was implemented to scale the servo angle proportionally to the steering error, reducing oscillation.

13. Future Enhancements

The current system provides a solid foundation for autonomous navigation. Several enhancements are planned for future iterations of this project:

- **Multi-Waypoint Navigation:** Extension of the firmware to support a sequence of GPS waypoints, enabling the car to follow a pre-planned route with multiple stops.
- **Wi-Fi/Bluetooth Remote Monitoring:** Leveraging the ESP32's built-in Wi-Fi to stream live GPS coordinates, compass heading, and sensor readings to a web dashboard for real-time monitoring.
- **PID Steering Controller:** Replacing the current proportional controller with a full PID controller to improve heading tracking accuracy and reduce oscillation.
- **Camera Integration:** Addition of an ESP32-CAM module for live video streaming and future implementation of computer vision-based obstacle detection.
- **Rechargeable LiPo Battery System:** Replacing the 12V batteries with a LiPo battery pack and a dedicated BMS (Battery Management System) for safer, lighter power management.
- **RTK GPS Upgrade:** Upgrading to an RTK (Real-Time Kinematic) GPS module to achieve centimetre-level positioning accuracy for precision navigation tasks.

14. Conclusion

This project successfully demonstrates the design and implementation of an autonomous GPS-guided smart car using the ESP32 microcontroller, GPS M8N module with compass, three ultrasonic sensors, a servo motor, and an L298N motor driver. The vehicle is capable of navigating from one geographic coordinate to another without any manual intervention, leveraging the Haversine formula for distance calculation and compass-based bearing correction for directional control.

The integration of multiple IoT components — sensor fusion, real-time processing, actuation, and autonomous decision-making — on a compact mobile platform represents a significant achievement for an undergraduate IoT project. The system achieved reliable navigation accuracy within the expected GPS error margin, effective obstacle detection, and stable heading correction.

This project has provided the team with practical hands-on experience in embedded systems programming, hardware interfacing, sensor integration, and navigation algorithm implementation. It reinforces the potential of IoT technology to enable intelligent, autonomous machines capable of operating in the real world.

15. Team Contributions

The project was a collaborative effort by five students, each contributing to specific areas of the design, development, and testing process:

S.No.	Student Name	Role & Contribution
1	Himanshi	Team Lead; GPS & Compass module integration; coordinate navigation algorithm
2	Piyish Kumar	ESP32 firmware development; motor driver control and PWM programming
3	Vanshika Sharma	Hardware assembly; circuit wiring; breadboard and component connections
4	Priyanka Singh	Ultrasonic sensor integration; obstacle detection and avoidance logic
5	Ishika Dutta	Testing & validation; report writing; documentation and presentation

Table 3: Team Member Contributions

16. References

- [1] u-blox AG. (2023). NEO-M8N GPS Module Datasheet. u-blox AG, Switzerland.
- [2] Honeywell International Inc. (2013). HMC5883L 3-Axis Digital Compass IC Datasheet. Honeywell Sensing and Control.
- [3] Espressif Systems. (2023). ESP32 Technical Reference Manual, Version 5.1. Espressif Systems Co., Ltd.
- [4] Sinnott, R. W. (1984). Virtues of the Haversine. *Sky and Telescope*, 68(2), 159.
- [5] HC-SR04 Ultrasonic Sensor Datasheet. (2013). Cytron Technologies Sdn. Bhd.
- [6] Arduino Community. (2024). ESP32 Arduino Core Documentation. GitHub: [espressif/arduino-esp32](https://github.com/espressif/arduino-esp32).
- [7] Mikal Hart. (2023). TinyGPS++ Library Documentation. Available at: <https://github.com/mikalhart/TinyGPSPlus>
- [8] L298N Motor Driver Datasheet. (2000). STMicroelectronics, Application Note.
- [9] Kaplan, E. D., & Hegarty, C. J. (2017). *Understanding GPS/GNSS: Principles and Applications* (3rd ed.). Artech House.

[10] Siegwart, R., Nourbakhsh, I. R., & Scaramuzza, D. (2011). Introduction to Autonomous Mobile Robots (2nd ed.). MIT Press.

— End of Report —